

Since each `Monster` object remembers its location and speed, in the `move()` method, each `Monster` can move to and remember its new location.

This technique of building a list of objects and calling the same method of all objects in the list is extremely useful, and it's a standard approach to dealing with a collection of similar objects. We will use this approach quite often when we get to building games using pygame later.

Interface vs. Implementation

Our earlier `Account` class seems to have methods and instance variables that work well. When you're confident your code is working well, you no longer have to be concerned with the details within the class. When a class does what you want it to do, all you need to remember is what methods are available in the class. There are two different ways to look at a class: by focusing on what it is capable of doing (the *interface*) and how it works internally (the *implementation*).

interface

The collection of methods a class provides (and the parameters that each method expects). The interface shows *what* an object created from the class can do.

implementation

The actual code of the class, which shows *how* an object does what it does.

If you are the creator or maintainer of a class, you need to fully understand the implementation—the code of all of the methods and how they work together to affect the instance variables. If you are purely writing code to *use* a class, you only need to concern yourself with the interface—the different methods that are available in the class, the values that need to be passed into each, and any value(s) that are returned from the methods. If